

Chương 3

Thao tác dữ liệu

Nội dung

- ❑ Truy vấn dữ liệu với lệnh SELECT
 - 1. Kết nối dữ liệu
 - 2. Truy vấn con
 - 3. Thống kê dữ liệu
- ❑ Nhóm lệnh DML: INSERT, UPDATE, DELETE
- ❑ Vai trò của View: cách tạo và sử dụng

Cú pháp lệnh Select

**SELECT [ALL | DISTINCT] [TOP n [WITH
TIES]] select_list
[INTO new_table]
FROM table_source
[WHERE search_condition]
[GROUP BY group_by_expression]
[HAVING search_condition]
[ORDER BY order_expression [ASC |
DESC]]**

- ☐ *ORDER BY : Sắp xếp*
- ☐ *WHERE: Điều kiện*
- ☐ *GROUP BY: Nhóm*
- ☐ *HAVING: Điều kiện nhóm*

Ví dụ lệnh SELECT

The diagram illustrates the components of a SQL SELECT statement. The statement is written in yellow text: `SELECT empno, ename, sal, deptno`, `FROM emp, dept`, `WHERE deptno=30`, and `ORDER BY ename`. Each part is enclosed in a black oval, and an arrow points from the oval to its corresponding label in dark gray text. The labels are: 'Column names' for the SELECT clause, 'Table names' for the FROM clause, 'Condition' for the WHERE clause, and 'Sort order' for the ORDER BY clause. The entire diagram is set against a light blue oval background.

SELECT empno, ename, sal, deptno → Column names

FROM emp, dept → Table names

WHERE deptno=30 → Condition

ORDER BY ename → Sort order

Truy vấn đơn giản

- Chọn tất cả các cột trong một bảng

Syntax

```
SELECT * FROM < table name >
```

Example

```
SELECT * FROM [Khach Hang]
```

Truy vấn đơn giản

- Chọn một vài cột trong một bảng

Syntax

```
SELECT <column1>, <column2>  
FROM <table name>
```

Example

```
SELECT Masp, Tensp FROM [San Pham]
```

Truy vấn đơn giản

- Kết nối các cột thành một cột

Syntax

```
SELECT <column1>+<'constant' >  
FROM <table name>
```

Example

```
SELECT HoNV+ ' ' + TenNv  
FROM [Nhan vien]
```

Truy vấn đơn giản

- Đặt tên cho cột mới

Syntax

```
SELECT <column1> as <'alias'>  
FROM <table name>
```

Example

```
SELECT Honv + ' ' + Tennv AS 'HOTEN' FROM  
[Nhan vien]
```


Truy vấn đơn giản

- Tạo cột tính toán

Example

```
SELECT Mahd, Soluong*Dongia AS 'Tong Tien'  
FROM [Chi Tiet Hoa Don]
```

Truy vấn đơn giản

- Loại bỏ những dòng trùng nhau

Syntax

```
SELECT DISTINCT <column1>  
FROM <table name>
```

Example

```
SELECT DISTINCT Makh  
FROM [hoa don]
```

```
SELECT diadiem  
FROM DIADIEM_PHG
```

diadiem
TP HCM
HA NOI
TP HCM

```
SELECT DISTINCT diadiem  
FROM DIADIEM_PHG
```

diadiem
TP HCM
HA NOI

Truy vấn đơn giản

- Chỉ có n hàng đầu tiên hay n% của các hàng của bảng kết quả được xuất

Syntax

```
SELECT TOP n [PERCENT] <column name>  
FROM <table name>
```

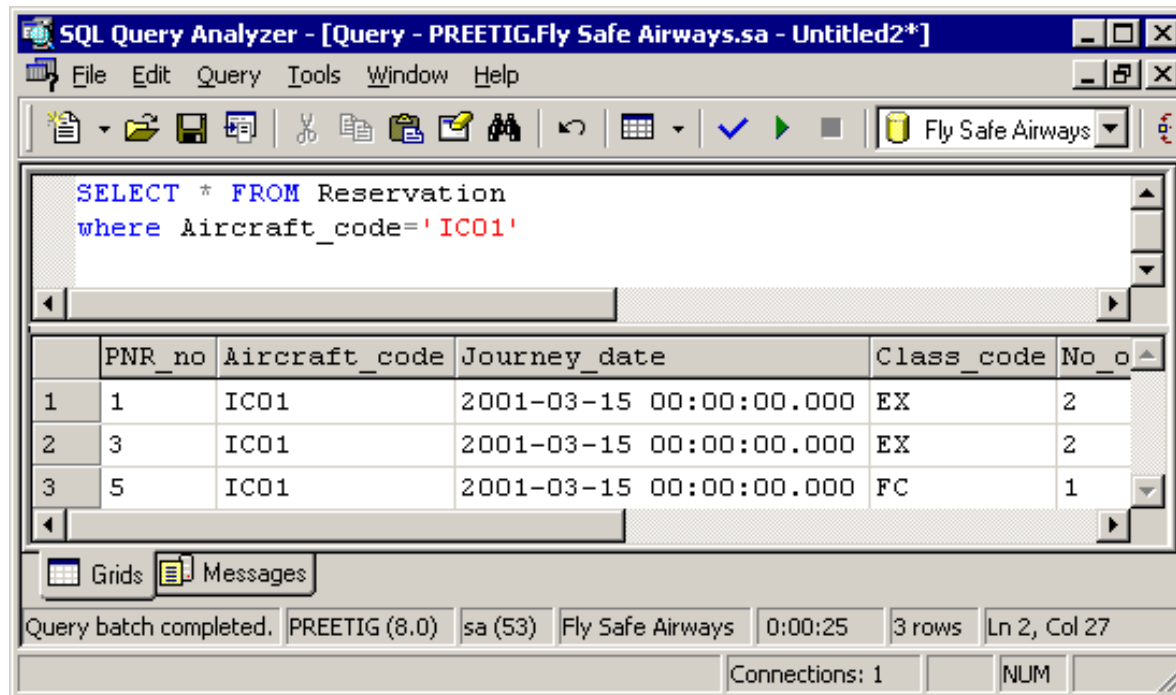
Examples

```
SELECT TOP 3 masp, dongia FROM [Chi tiet  
hoa don]
```

```
SELECT TOP 4 PERCENT masp FROM [San  
pham]
```

Mệnh đề WHERE

- Chứa điều kiện lọc dữ liệu cần trả về
- Cú pháp: **WHERE** <search_condition>



Phép toán quan hệ - Relational Operators

Operator	Meaning
=	Equal To
>	Greater Than
<	Less than
>=	Greater Than or Equal To
<=	Less Than or Equal To
!=	Not

Phép toán Logical

- Phép toán và (AND operator)

Example

```
SELECT Mahd, NgayLapHD, Makh  
FROM [Hoa don]  
WHERE Month(NgayLapHD) = 3 AND  
Year(NgayLapHD)=2012
```

Phép toán Logical

- Phép toán hoặc (OR operator)

Example

```
SELECT * FROM [Hoa don]  
WHERE Makh = 'FISC' OR Makh = 'HUNSAN'
```

Phép toán Logical

- Phép toán phủ định (NOT operator)

Example

```
SELECT * FROM [Hoa don]  
WHERE NOT Manv= 10
```


Các toán tử SQL

- **LIKE**: giống 1 chuỗi
- **IS NOT NULL**: không phải giá trị rỗng
- **BETWEEN ... AND ...**: giữa 2 giá trị
- **IN**: đặt giá trị trong 1 danh sách
- **ALL/ ANY (SOME)**: được dùng trong lệnh truy vấn con và kết quả là nhiều dòng.
 - <ANY: Nhỏ hơn trị cao nhất
 - >ANY: Lớn hơn trị thấp nhất
 - =ANY: Tương đương với IN
 - >ALL: Lớn hơn trị cao nhất
 - <ALL: Nhỏ hơn trị thấp nhất
- **EXIST**: kiểm tra sự tồn tại của 1 dữ liệu trong nhiều dòng.

Wildcard Characters

(Các ký tự thay thế)

Wildcard	Description	Example
_	Represents a single character	SELECT Meal_Code FROM Meal WHERE Meal_Code LIKE 'C_'
%	Represents a string of any length	SELECT Meal_Code FROM Meal WHERE Meal_Code LIKE 'CO_%'
[]	Represents a single character within the range enclosed in the brackets	SELECT * FROM flight WHERE aircraft_code LIKE '9W0[1-2]'
[^]	Represents any single character not within the range enclosed in the brackets	SELECT * FROM flight WHERE aircraft_code LIKE '9W0[^1-2]'

Các Hàm - Functions



- AVG()
- MIN()
- MAX()
- SUM()
- COUNT()
- SQUARE()
- SQRT()
- ROUND()

- ASCII()
- CHAR()
- UPPER()
- LOWER()
- LEN()
- LTRIM()
- RTRIM()
- LEFT()
- RIGHT()

- GETDATE()
- DATEPART(YY,getdate())
- DATEDIFF(X,Y,Z)
- DAY(),MONTH(),YEAR()

Các Hàm - Functions

	Function	Description
General Functions	ISDATE(exp)	Returns 1 if exp is a valid date
	ISNULL(exp1,exp2)	Returns Null if exp1 is NULL, otherwise exp1 returned
	ISNUMERIC(exp)	Returns 1 if exp is a number type
String Functions	ASCII(char)	Returns the ASCII value of a Character.
	CHAR(int)	Returns the character value for an ASCII integer value.
	CHARINDEX(string1, string2, start)	Returns the starting position for string1 in string2 optionally starting at position start.

Các Hàm - Functions

	Function	Description
String Functions	NCHAR(int)	Returns the UNICODE character represented by int.
	LEN(string)	Returns the length of the string.
	LOWER(string)	Returns the string passed in with all characters converted to lowercase.
	UPPER(string)	Returns the string passed in with all characters converted to uppercase.

Các Hàm - Functions

	Function	Description
String Functions	REVERSE(string)	Returns the reverse of a character expression.
	RIGHT(string, int)	Returns the int number of characters from the right side of the string.
	LEFT(string, int)	Returns the first int characters from String.

Các Hàm - Functions

	Function	Description
String Functions	SUBSTRING(string, start, int)	Returns a portion of the string string starting at position start and continuing for int characters.
	RTRIM(string)	Returns the string with all blank spaces from the end of the string Removed.
	LTRIM(string)	Returns the string with all blank spaces from the left side of the string removed.

Các Hàm - Functions

	Function	Description
String Functions	SPACE(int)	Returns int number of spaces.
	STR(float, length, decimal)	Converts a numeric value to a string.
	STUFF(string, start, length, char)	Removes length characters from string starting with character start and replaces them with char.

Các Hàm - Functions

	Function	Description
String Functions	UNICODE(Unicode string)	Returns the numeric value of the first character of a UNICODE Expression.

Các Hàm - Functions

	Function	Description
Date and Time Functions	DATEPART(day/month/..., day)	Returns the specific part of the date as an integer.
	DAY(date)	Returns the numeric day of the week for date.

Các Hàm - Functions

	Function	Description
Date and Time Functions	GETDATE()	GETDATE() Returns the current server date and <i>time</i> .
	MONTH(<i>date</i>)	Returns the numeric month number of <i>date</i> .
	YEAR (<i>date</i>)	Returns the numeric year number of <i>date</i> .

So sánh Chuỗi

- ❑ So sánh gần đúng sử dụng “like”
 - Hai ký tự thay thế: ‘_’ và ‘%’
- ❑ Tìm tất cả các mã nhân viên có địa chỉ ở quận 1.

```
SELECT *  
FROM [Nhan vien]  
WHERE diachi LIKE '%Q1'
```

Cho biết tên nhân viên sinh vào những năm 1960

```
SELECT Honv, Tennv, NgaySinh  
FROM [Nhan vien]  
WHERE convert(char(8), NgaySinh,1) like '____6_'
```

Các thuộc tính Trùng tên

tuple variable

Biến bộ

- ❑ Cho biết hai nhân viên có cùng thành phố

```
SELECT  NV1.TenNv, NV2.TenNv, NV1.ThanhPho,  
        NV2.Thanhpho  
FROM    [Nhan vien] NV1, [Nhan vien] NV2  
WHERE   NV1.Thanhpho=NV2.Thanhpho
```

- ❑ Có thể sử dụng biến bộ bất kỳ lúc nào để thuận tiện và dễ đọc!

```
SELECT  NV1.TenNv, NV2.TenNv, NV1.ThanhPho,  
        NV2.Thanhpho  
FROM    [Nhan vien] NV1, [Nhan vien] NV2  
WHERE   NV1.Thanhpho=NV2.Thanhpho
```

Truy vấn từ nhiều bảng & Where (điều kiện kết nối)

- ❑ Danh sách các hoá đơn gồm Mahd, Tenkh

```
SELECT      Mahd, [Hoa Don].Makh, Tenkh
FROM        [Hoa don], [Khach hang]
WHERE       [Hoa don].Makh= [Khach hang].Makh
```

- ❑ Danh sách các hoá đơn do nhân viên có tên bắt đầu là D lập

```
SELECT      [Hoa don].Mahd, Honv + ' '+Tennv as Hoten
FROM        [Nhan vien], [Hoa don]
WHERE       [Nhan vien].Manv = [Hoa don].Manv And TenNV
           like 'D%'
```

- ❑ Danh sách các hoá đơn do nhân viên có Thành phố là HCM lập

```
SELECT      O.Mahd, Honv + ' '+Tennv as HoTen
FROM        [Nhan vien] E, [Hoa don] O
WHERE       E.Manv =O.Manv And Thanhpho ='HCM'
```

Sắp xếp - ORDER BY Clause

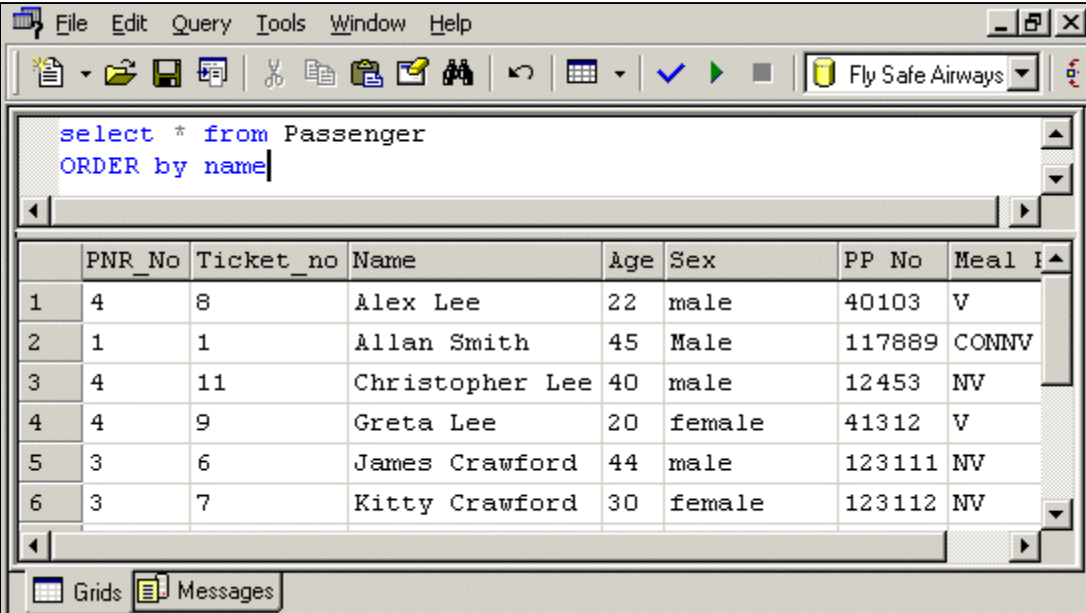
Xác định thứ tự của bộ kết quả

Cú pháp

[ORDER BY { *order_by_expression* [ASC | DESC] } [,...n]]

ASC (ascending) : xếp theo thứ tự tăng

DESC (descending): xếp theo thứ tự giảm



The screenshot shows a database query tool window. The title bar includes 'File', 'Edit', 'Query', 'Tools', 'Window', and 'Help'. The menu bar includes 'File', 'Edit', 'Query', 'Tools', 'Window', and 'Help'. The toolbar includes icons for file operations, query execution, and a dropdown menu showing 'Fly Safe Airways'. The query editor contains the following SQL query:

```
select * from Passenger
ORDER by name
```

The results are displayed in a table with the following columns: PNR_No, Ticket_no, Name, Age, Sex, PP No, and Meal I. The data is sorted by Name in ascending order.

	PNR_No	Ticket_no	Name	Age	Sex	PP No	Meal I
1	4	8	Alex Lee	22	male	40103	V
2	1	1	Allan Smith	45	Male	117889	CONNV
3	4	11	Christopher Lee	40	male	12453	NV
4	4	9	Greta Lee	20	female	41312	V
5	3	6	James Crawford	44	male	123111	NV
6	3	7	Kitty Crawford	30	female	123112	NV

The bottom of the window has a status bar with 'Grids' and 'Messages' buttons.

Nhóm dữ liệu trong bảng kết quả

- Những mệnh đề dùng để nhóm dữ liệu trong bảng kết quả:
 - **GROUP BY**: tổng hợp bảng kết quả theo nhóm bằng cách dùng các hàm gộp
 - **COMPUTE** và **COMPUTE BY**: mệnh đề **COMPUTE** trong lệnh **SELECT** được dùng để phát ra các hàng tổng hợp bằng cách dùng hàm gộp. Mệnh đề **COMPUTE BY** được dùng để tổng hợp thêm các hàng kết quả theo cột

Mệnh đề GROUP BY

- Cú pháp:

[GROUP BY [ALL] *group_by_expression* [,...n]]

bảng kết quả sẽ chứa tất cả các nhóm kể cả những nhóm không thỏa mãn điều kiện lọc trong mệnh đề WHERE, những nhóm không thỏa điều kiện sẽ có giá trị null.

- *group_by_expression*: biểu thức dùng để xác định cột được nhóm

Mệnh đề GROUP BY

- Ví dụ:

```
SELECT Mahd, SUM(Soluong* Dongia)  
AS 'Thanh tien' FROM [Chi tiet hoa don]  
GROUP BY mahd
```

```
SELECT Mahd, AVG(Soluong * DonGia)  
AS 'Trung Binh' FROM [Chi Tiet hoa don]  
GROUP BY Mahd
```

Mệnh đề GROUP BY

- Ví dụ:

```
SELECT Mahd, MIN(Soluong * Dongia)
AS 'Thanh tien nho nhat' FROM [Chi Tiet hoa don]
GROUP BY Mahd
```

```
SELECT Mahd, MAX(Soluong * Dongia)
AS 'Thanh Tien Lon Nhat' FROM [Chi Tiet Hoa Don]
GROUP BY Mahd
```

Mệnh đề GROUP BY

- Ví dụ:

```
SELECT Count(Mahd)  
AS 'So Hoa Don' FROM [Hoa don]
```

```
SELECT Count(*)  
AS 'So Hoa Don' FROM [Hoa Don]
```

```
SELECT Makh, Count(Mahd)  
AS 'So HD cua tung khach hang' FROM [Hoa don]  
GROUP BY Makh
```

Mệnh đề GROUP BY

- Ví dụ:

```
SELECT Masp, Sum(Soluong) As Total  
FROM [Chi Tiet Hoa Don]  
WHERE Masp=2  
GROUP BY Masp
```

```
SELECT Makh, Count(Mahd)  
AS 'So HD cua khach hang' FROM [Hoa don]  
WHERE Makh like '%o'  
GROUP BY Makh
```

GROUP BY và HAVING

- Có thể hạn chế các nhóm trong bảng kết quả bằng mệnh đề HAVING.
- Chỉ sau khi dữ liệu đã được nhóm và tổng hợp, điều kiện trong mệnh đề HAVING mới được áp dụng.
- Không thể dùng 1 cột mà nó không tham gia vào hàm gộp của mệnh đề SELECT hay của mệnh đề GROUP BY.
- **SELECT Masp, AVG(Dongia) FROM [San pham] GROUP BY Masp HAVING (AVG(Dongia) > 10)**

Sử dụng WHERE và HAVING

- Mệnh đề HAVING giống như mệnh đề WHERE nhưng chỉ áp dụng cho cả nhóm trong khi mệnh đề WHERE áp dụng cho từng hàng.
- Một truy vấn có thể chứa cả mệnh đề WHERE và mệnh đề HAVING.
 - Mệnh đề WHERE được áp dụng trước cho các hàng trong bảng được truy vấn. Chỉ những hàng nào thoả mãn điều kiện của mệnh đề WHERE mới được nhóm dữ liệu.
 - Sau đó mệnh đề HAVING sẽ được áp dụng cho các nhóm. Chỉ những nhóm thoả mãn điều kiện HAVING mới được xuất ra bảng kết quả.

Sử dụng WHERE và HAVING

Ví dụ

```
SELECT Masp, Sum(Soluong) As Total  
FROM [Chi tiet hoa don]  
GROUP BY Masp  
HAVING Sum(Soluong)>=30
```

```
SELECT Makh, Count(Mahd)  
AS 'So hoa don cua KH' FROM [Hoa don]  
GROUP BY Makh  
HAVING Count(Mahd)<=5
```


Mệnh đề COMPUTE

Thường dùng để kiểm tra số liệu, dùng kèm với các hàm thống kê SUM, AVG, MAX, MIN,...

Ví dụ

```
SELECT Masp, Mahd, Soluong  
FROM [Chi Tiet Hoa Don]  
ORDER BY Masp, Mahd  
COMPUTE Sum(Soluong)
```

```
SELECT Masp, Mahd, Soluong As Total  
FROM [Chi Tiet Hoa Don]  
ORDER BY Masp, Mahd  
COMPUTE SUM(Soluong) By Masp  
COMPUTE SUM(Soluong)
```

Mệnh đề COMPUTE

❑ COMPUTE...BY...: Có kết nhóm

1) SELECT c.Makh, o.Mahd, (od.Soluong * od.Dongia) as
‘Total’ FROM [Hoa don] o, [Chi Tiet hoa don] od,[Khach
hang] c

WHERE c.Makh = o.Makh AND o.Mahd = od.Mahd AND
c.Makh LIKE ‘T%’ ORDER BY c.Makh COMPUTE SUM(
od.Soluong * od.Dongia)

2) SELECT c.Makh, o.Mahd, (od.Soluong * od.Dongia) as
‘Tong’ FROM [Hoa don] o, [Chi Tiet Hoa Don] od,
[Khach hang] c

WHERE c.Makh = o.Makh AND o.Mahd = od.Mahd AND
c.Makh LIKE ‘T%’ ORDER BY c.Makh COMPUTE
SUM(od.soluong * od.dongia) BY c.Makh

Mệnh đề COMPUTE

- ❑ Khi sử dụng mệnh đề COMPUTE ... BY cần tuân theo các qui tắc dưới đây:
 - Từ khóa DISTINCT không cho phép sử dụng với các hàm gộp dòng
 - Hàm COUNT(*) không được sử dụng trong COMPUTE.
 - Sau COMPUTE có thể sử dụng nhiều hàm gộp, khi đó các hàm phải phân cách nhau bởi dấu phẩy.
 - Các cột sử dụng trong các hàm gộp xuất hiện trong mệnh đề COMPUTE phải có mặt trong danh sách chọn.
 - Không sử dụng SELECT INTO trong một câu lệnh SELECT có sử dụng COMPUTE.
 - Nếu sử dụng mệnh đề COMPUTE ... BY thì cũng phải sử dụng mệnh đề ORDER BY. Các cột liệt kê trong COMPUTE ... BY phải giống hệt hay là một tập con của những gì được liệt kê sau ORDER BY. Chúng phải có cùng thứ tự từ trái qua phải, bắt đầu với cùng một biểu thức và không bỏ qua bất kỳ một biểu thức nào.

Mệnh đề JOIN-Kết nối nhiều bảng

- Mệnh đề join dùng để kết nối dữ liệu từ nhiều hơn 1 bảng
- Cú pháp

SELECT column_name [,n..]

FROM table_name table_alias

[CROSS|INNER|[LEFT | RIGHT]OUTER] JOIN
table_name table_alias

[ON table_name.ref_column_name
join_operator table_name.ref_column_name]

[WHERE search_condition]

Kết nối các bảng

- Kết nối chỉ tồn tại trong thời gian truy vấn.
- Kết nối không thay đổi dữ liệu trong các bảng của cơ sở dữ liệu.
- Nên tạo bí danh (alias) cho tên bảng để tránh gõ tên dài và làm truy vấn dễ đọc hơn
- Ví dụ

```
SELECT t.Mahd, NgaylapHD, Masp, Soluong, Dongia,  
       Thanhtien as Soluong*Dongia  
from [Chi tiet hoa don] t  
JOIN [Hoa don] h on t.mahd=h.mahd  
WHERE Makh='SJC'
```

Các cột tham gia kết nối

- Nếu kết nối nhiều hơn 2 bảng thì kết nối 2 bảng trước, sau đó kết nối nhóm này với bảng thứ ba.

- Ví dụ

```
SELECT o.Mahd,c.makh, p.Masp, Tensp,  
Soluong, o.dongia, Thanh tien =soluong*o.dongia  
FROM [Chi tiet Hoa don] o JOIN SanPham p  
ON o.Masp = p.Masp  
JOIN [Hoa don] c  
ON o.Mahd = c.Mahd
```

Các loại kết nối

- Inner Join
- Outer Join
- Cross Join
- Equi Join
- Natural Join
- Self Join

Kết nối nội - Inner joins

- Trong kết nối nội, dữ liệu từ nhiều bảng được hiển thị sau khi so sánh giá trị trong 1 cột chung. Chỉ những hàng mà có giá trị thoả mãn điều kiện kết nối trong cột chung đó mới được hiển thị.
- **Tích Cartesian**: việc kết nối nhiều bảng mà không có điều kiện kết nối trong mệnh đề ON sẽ tạo ra tích cartesian giữa 2 bảng
- Ví dụ:

```
SELECT t.Mahd, h.NgaylapHD, masp, Soluong, dongia,  
       Thanh tien as Soluong*Dongia from [Chi tiet hoa don] t  
JOIN [Hoa don] h on t.mahd=h.mahd
```


Kết nối nội - Inner joins

```
SELECT K.Makh, TenKH,  
       Mahd, NgaylapHD  
FROM [Khach hang] K INNER JOIN [Hoa don] h  
ON K.Makh = H.Makh
```

```
SELECT Tensp, c.Mahd, Soluong, dongia,  
       Soluong * DonGia AS [Thanhtien]  
FROM [San pham] AS s INNER JOIN [Chi tiet hoa  
don] AS c ON s.Masp = c.Masp  
INNER JOIN [Hoa don] As h  
ON h.mahd =c.mahd  
WHERE Month(NgayLapHD) = 3
```

Kết nối nội với toán tử lớn hơn

Có thể thực hiện kết nối 2 bảng với điều kiện kết nối dùng toán tử không bằng nhau.

Ví dụ:

```
SELECT Tensp, c.Mahd, Soluong, c.dongia,  
       Soluong * c.DonGia AS [Thanh tien]  
FROM [San pham] AS s INNER JOIN [Chi tiet  
hoa don] AS c ON s.Masp > c.Masp
```

Kết nối nội – Dùng where

Bảng DONVI

MADV	TENDV
1	Doi ngoai
2	Hanh chinh
3	Ke toan
4	Kinh doanh

Bảng NHANVIEN

HOTEN	MADV
Thanh	1
Hoa	2
Nam	2
Vinh	1
Hung	5
Phuong	NULL

Câu lệnh:

```
SELECT *  
FROM nhanvien,donvi  
WHERE nhanvien.madv=donvi.madv
```

có kết quả là:

HOTEN	MADV	MADV	TENDV
Thanh	1	1	Doi ngoai
Hoa	2	2	Hanh chinh
Nam	2	2	Hanh chinh
Vinh	1	1	Doi ngoai

Kết nối ngoại - Outer joins

- Kết nối ngoại được dùng để cho ra kết quả chứa tất cả các hàng của 1 bảng và các hàng trùng nhau của bảng còn lại. Những cột mà không có giá trị phù hợp sẽ được hiển thị giá trị NULL.

- *Cú pháp*

**SELECT column_name, column_name
[,column_name]**

**FROM table_name [LEFT | RIGHT] OUTER JOIN
table_name**

ON table_name.ref_column_name

join_operator table_name.ref_column_name

Kết nối ngoài - Outer joins

SQL cung cấp các loại phép nối ngoài sau đây:

- **Phép nối ngoài trái** (ký hiệu: *=): Phép nối này hiển thị trong kết quả truy vấn tất cả các dòng dữ liệu của bảng nằm bên trái trong điều kiện nối cho dù những dòng này không thoả mãn điều kiện của phép nối
- **Phép nối ngoài phải** (ký hiệu: =*): Phép nối này hiển thị trong kết quả truy vấn tất cả các dòng dữ liệu của bảng nằm bên phải trong điều kiện nối cho dù những dòng này không thoả điều kiện của phép nối.

Kết nối ngoại – Left Outer joins

Bảng DONVI

MADV	TENDV
1	Doi ngoai
2	Hanh chinh
3	Ke toan
4	Kinh doanh

Bảng NHANVIEN

HOTEN	MADV
Thanh	1
Hoa	2
Nam	2
Vinh	1
Hung	5
Phuong	NULL

Nếu thực hiện phép nối ngoại trái giữa bảng NHANVIEN và bảng DONVI:

```
SELECT *  
FROM nhanvien,donvi  
WHERE nhanvien.madv*=donvi.madv
```

kết quả của câu lệnh sẽ là:

HOTEN	MADV	MADV	TENDV
Thanh	1	1	Doi ngoai
Hoa	2	2	Hanh chinh
Nam	2	2	Hanh chinh
Vinh	1	1	Doi ngoai
Hung	5	NULL	NULL
Phuong	NULL	NULL	NULL

Kết nối trái - LEFT OUTER JOIN

- Tất cả các hàng từ bảng bên trái trong mỗi kết nối giữa 2 bảng sẽ được hiển thị trong bảng kết quả.
- **Ví dụ:**

```
SELECT Manv, Hoten, cv, d.macv,tench  
FROM Nhanvien n INNER JOIN ChucVu d ON  
n.cv=d.macv
```

```
SELECT Manv, Hoten, cv, d.macv,tench  
FROM Nhanvien n LEFT OUTER JOIN ChucVu d ON  
n.cv=d.macv
```

Kết nối ngoại – Right Outer joins

Bảng DONVI

MADV	TENDV
1	Doi ngoai
2	Hanh chinh
3	Ke toan
4	Kinh doanh

Bảng NHANVIEN

HOTEN	MADV
Thanh	1
Hoa	2
Nam	2
Vinh	1
Hung	5
Phuong	NULL

Và kết quả của phép nối ngoài phải:

```
select *  
from nhanvien,donvi  
where nhanvien.madv=*donvi.madv
```

như sau:

HOTEN	MADV	MADV	TENDV
Thanh	1	1	Doi ngoai
Vinh	1	1	Doi ngoai
Hoa	2	2	Hanh chinh
Nam	2	2	Hanh chinh
NULL	NULL	3	Ke toan
NULL	NULL	4	Kinh doanh

Kết nối trái - LEFT OUTER JOIN

- Tất cả các hàng từ bảng bên trái trong mỗi kết nối giữa 2 bảng sẽ được hiển thị trong bảng kết quả.
- **Ví dụ:**

```
SELECT Manv, Hoten, cv, d.macv, tench  
FROM Nhanvien n RIGHT OUTER JOIN ChucVu d  
ON n.cv=d.macv
```

```
SELECT Manv, Hoten, cv, d.macv, tench  
FROM Nhanvien n FULL JOIN ChucVu d ON  
n.cv=d.macv
```

Kết nối đầy đủ - FULL OUTER JOIN

Bảng DONVI

MADV	TENDV
1	Doi ngoai
2	Hanh chinh
3	Ke toan
4	Kinh doanh

Bảng NHANVIEN

HOTEN	MADV
Thanh	1
Hoa	2
Nam	2
Vinh	1
Hung	5
Phuong	NULL

Ví dụ 2.33: Với hai bảng NHANVIEN và DONVI như ở trên, câu lệnh

```
SELECT *  
FROM nhanvien FULL OUTER JOIN donvi  
ON nhanvien.madv=donvi.madv
```

cho kết quả là:

HOTEN	MADV	MADV	TENDV
Thanh	1	1	Doi ngoai
Hoa	2	2	Hanh chinh
Nam	2	2	Hanh chinh
Vinh	1	1	Doi ngoai
Hung	5	NULL	NULL
Phuong	NULL	NULL	NULL
NULL	NULL	4	Kinh doanh
NULL	NULL	3	Ke toan

Cross join (kết nối chéo)

- Cross join trả về mọi tổ hợp có thể có của tất cả các hàng trong các bảng kết nối.
- Cross join không có mệnh đề ON
 - Nếu không mệnh đề WHERE, cross join sẽ tạo ra **tích Cartesian**
 - Nếu có mệnh đề WHERE, cross join sẽ thực hiện như 1 kết nối nội

Kết nối chéo - Cross join

- Ví dụ 1:

```
SELECT [San pham].Masp, tensp, soluong,  
c.dongia FROM [San pham] CROSS JOIN [Chi tiet  
hoa don]
```

- Ví dụ 2:

```
SELECT [San pham].Masp, tensp, soluong, dongia  
FROM [San pham] CROSS JOIN [Chi tiet hoa don]  
as c WHERE [San pham].masp = c.Masp
```

Truy vấn tự kết nối (self-join)

- Ví dụ: tìm tất cả các khách hàng mua ít nhất 2 đơn hàng

```
SELECT t1.Makh, t1.MAHD,t2.MAHD  
FROM [Hoa don] t1 JOIN [Hoa don] t2  
ON t1.Mahd!= t2.Mahd AND  
t1.Makh = t2.Makh
```

$$\pi_{StudID}(\sigma_{CrsCode \neq CrsCode2}(
TRANSCRIPT > < TRANSCRIPT
[StudID, CrsCode2, Semester2, Grade2]))$$

Truy vấn select - merge

Chức năng mới trong SQL2008 là phát biểu merge. Bạn có thể trộn 2 hay nhiều bảng

```
SELECT ltr.FName, lt.LectureDay, lt.LectureTime
FROM dbo.LECTURE lt
inner merge JOIN dbo.LECTURER ltr
on lt.StaffNO=ltr.StaffNO
ORDER BY ltr.FName
```

	FName	LectureDay	LectureTime
1	Francisco	Wednesday	2008-04-30 00:00:00.000
2	Patricio	Monday	2008-05-13 00:00:00.000
3	Patricio	Monday	2008-04-15 00:00:00.000
4	Pedro	Tuesday	2008-04-20 00:00:00.000
5	Roland	Friday	2008-09-14 00:00:00.000
6	Sven	Monday	2008-05-23 00:00:00.000
7	Sven	Wednesday	2008-07-13 00:00:00.000
8	Victoria	tuesday	2008-03-25 00:00:00.000
9	Yang	Friday	2008-02-15 00:00:00.000

Truy vấn với giá trị Null

Phép nối và các giá trị NULL

Nếu trong các cột của các bảng tham gia vào điều kiện của phép nối có các giá trị NULL thì các giá trị NULL được xem như là không bằng nhau.

Ví dụ: Giả sử ta có hai bảng TABLE1 và TABLE2 như sau:

TABLE1

A	B
1	b1
NULL	b2
4	b3

TABLE2

C	D
NULL	d1
4	d2

Câu lệnh:

```
SELECT *  
FROM table1, table2  
WHERE A != C
```

Có kết quả là:

A	B	C	D
1	b1	NULL	NULL
NULL	b2	NULL	NULL
4	b3	4	d2

Truy vấn con - Subqueries

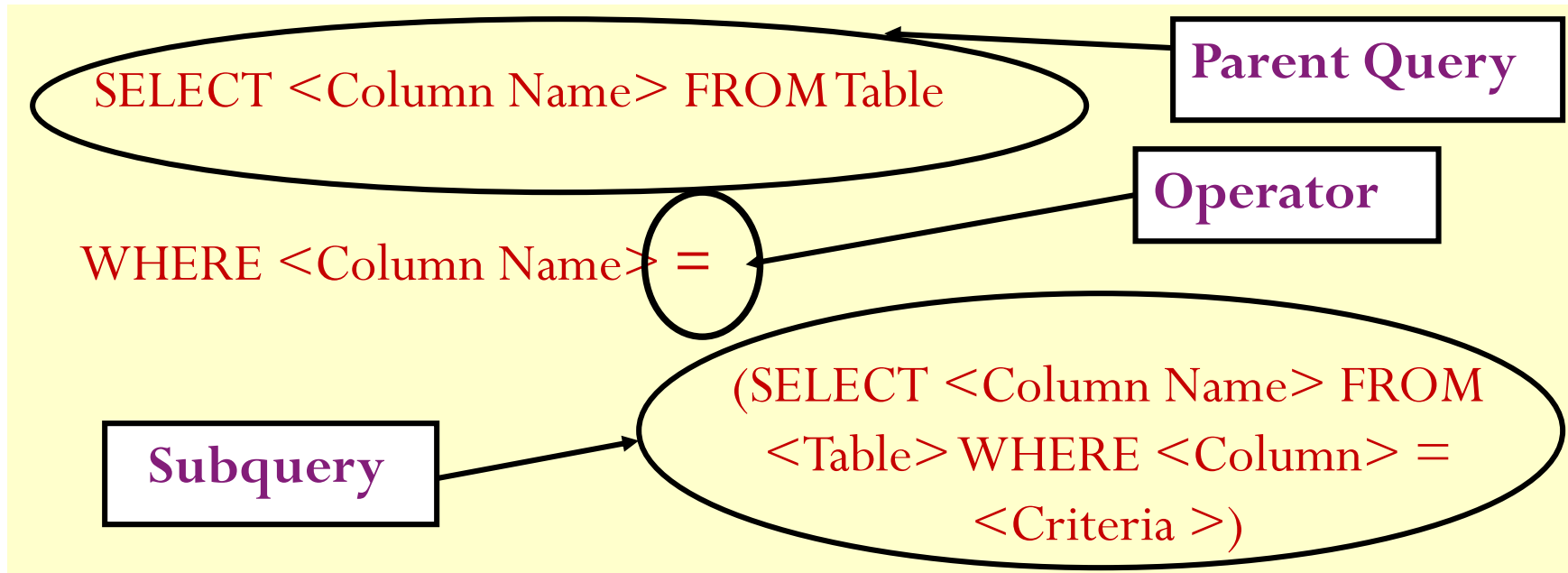
- Subquery là lệnh SELECT mà kết quả trả về là 1 giá trị đơn (single value) và được đặt lồng vào bên trong các lệnh SELECT, INSERT, UPDATE, hay DELETE, hay bên trong truy vấn con khác.
- Subquery có thể được dùng bất kỳ nơi nào mà biểu thức được phép dùng

Truy vấn con - Subqueries

A subquery is a SELECT statement inside another SELECT statement.



Cú pháp



Truy vấn con - Subqueries

Khi sử dụng truy vấn con cần lưu ý một số quy tắc sau:

- Một truy vấn con phải được viết trong cặp dấu ngoặc. Trong hầu hết các trường hợp, một truy vấn con thường phải có kết quả là một cột (tức là chỉ có duy nhất một cột trong danh sách chọn).
- Mệnh đề COMPUTE và ORDER BY không được phép sử dụng trong truy vấn con.
- Các tên cột xuất hiện trong truy vấn con có thể là các cột của các bảng trong truy vấn ngoài.
- Một truy vấn con thường được sử dụng làm điều kiện trong mệnh đề WHERE hoặc HAVING của một truy vấn khác.
- Nếu truy vấn con trả về đúng một giá trị, nó có thể sử dụng như là một thành phần bên trong một biểu thức (chẳng hạn xuất hiện trong một phép so sánh bằng)

Truy vấn con - Subqueries

- TH1: Dùng để lấy các field

```
SELECT h.Mahd, h.NgayLapHD, (SELECT  
    MAX(C.Dongia)  
    FROM [Chi tiet hoa don] AS c  
    WHERE h.Mahd =c.Mahd) AS MaxUnitPrice  
FROM [Hoa don] AS h
```

- Subquery có thể được dùng để khôi phục dữ liệu từ nhiều bảng và có thể được dùng như 1 cách khác của join (alternative to a join)

Truy vấn con - Subqueries

- TH2: Phép so sánh đối với kết quả truy vấn con

VD: Cho biết những sản phẩm nào có đơn giá lớn hơn đơn giá của sản phẩm có tên bắt đầu bằng chữ N

SELECT Tensp **FROM** [San pham]

WHERE Dongia > (**SELECT** Dongia **FROM** [San pham] **WHERE** TenSp like 'N%')

- Cách 2: dùng join

SELECT P.Tensp **FROM** [San pham] **AS** p **JOIN** [San pham] **AS** P2

ON (P.Dongia > P2.Dongia)

WHERE P2.TenSp like 'N%'

Truy vấn con - Subqueries

- **TH2: Phép so sánh đối với kết quả truy vấn con**

VD: Cho biết những sản phẩm có tên bắt đầu bằng chữ N và đơn giá lớn hơn đơn giá của sản phẩm khác

```
SELECT Tensp FROM [San pham]  
WHERE TenSp like 'N%' And Dongia > (SELECT  
Dongia FROM [San pham])
```

- Cách 2: dùng join

```
SELECT P.Tensp FROM [San pham] AS p JOIN [San  
pham] AS P2  
ON (P.Dongia > P2.Dongia)  
WHERE P2.TenSp like 'N%'
```

Truy vấn con - Subqueries

- Subquery có thể được dùng theo 1 trong các dạng sau:
 - WHERE *expression* [NOT] IN (*subquery*)
 - WHERE *expression comparison_operator* [ANY | ALL] (*subquery*)
 - WHERE [NOT] EXISTS (*subquery*)

Subqueries với toán tử IN

TH3: Kết quả của subquery được dùng với IN (hay với NOT IN) thường là 1 danh sách (list) chứa từ 0 đến nhiều giá trị.

- Ví dụ:

```
SELECT Mahd, NgayLapHD FROM [Hoa don]
WHERE Makh IN (SELECT Makh FROM [Khach
hang] WHERE Diachi like '%Q1')
```

- Dùng cách 2 với mệnh đề join???

Subquery với các toán tử so sánh

- Các toán tử so sánh đơn giản (unmodified comparison operator)

=, < >, >, > =, <, ! >, ! <, or < =

- TH4: Subquery bắt đầu với toán tử so sánh đơn giản và trả về 1 giá trị đơn (single value)

```
SELECT Masp FROM [San pham] WHERE  
dongia > (SELECT MIN(Dongia) FROM [Chi tiet  
hoa don])
```


Subquery trả về một giá trị

- Use Aggregate Functions in Subquery as Max(), Min(), Avg(),...
- Khi một subquery được sử dụng trong mệnh đề SELECT, nó trả về dữ liệu thay cho một cột.
- Loại subquery này được gọi là **scalar** query bởi vì nó chỉ trả về một giá trị.

Ví dụ

```
SELECT Tensp, Dongia FROM [San pham]  
WHERE DVT = 'kg' and Dongia <  
      (SELECT AVG(Dongia) FROM [San pham])
```

Subquery trả về nhiều giá trị

- Sử dụng toán tử In, Not in, Any, All...

Ví dụ

```
1) SELECT Mahd FROM [Chi tiet hoa don]
   WHERE Masp In
   (SELECT Masp FROM [San Pham]
    WHERE DVT ='Kg')
```

```
2) SELECT Mahd, Masp, Soluong FROM [Chi
   tiet  hoa  don] WHERE Soluong >=All
   (SELECT Soluong FROM [Chi tiet hoa don])
```

Subquery trả về nhiều giá trị

- ❑ Cho biết các sản phẩm có đơn giá cao nhất

```
SELECT      *  
FROM        [San Pham]  
WHERE       Dongia >= ALL  
            (SELECT Dongia FROM [San pham])
```

- ❑ Cho biết các sản phẩm có đơn vị tính có chữ a và có đơn giá cao nhất

```
SELECT      *  
FROM        [San pham]  
WHERE       DVT like '%a%' and  
            Dongia >= ALL  
            (SELECT Dongia FROM [San pham])
```

Subquery với các toán tử so sánh

- **Subquery trả về một giá trị**

```
SELECT Tensp, Dongia FROM [San pham]  
WHERE Dongia < ( SELECT AVG(Dongia) FROM  
[San pham] WHERE Masp < 5)
```

- **Subquery trả về một dãy giá trị**

```
Select Mahd, s.MASP, Tensp from [SAN PHAM] s  
join [Chi tiet hoa don] c on s.MASP = c.MASP  
Where s.Masp IN (Select Masp  
From [Chi Tiet Hoa don]  
Where Masp < 5)
```

Subquery với các toán tử so sánh

- Toán tử so sánh phức (Modified Comparison operators) là toán tử so sánh đơn giản có kèm theo các từ All, any hay some

(=toán tử so sánh+ [All, Any, Some])

- Subquery bắt đầu với toán tử so sánh phức sẽ trả về 1 danh sách (list) của 0 hay nhiều giá trị và có thể bao gồm cả mệnh đề GROUP BY hay HAVING.

Subquery với các toán tử so sánh

- **>ALL** có nghĩa lớn hơn mọi giá trị.
- **>ANY** có nghĩa lớn hơn ít nhất 1 giá trị

Vd: >ALL (1, 2, 3) lớn hơn 3

>ANY (1, 2, 3) lớn hơn 1

- **Ví dụ:**

```
SELECT Manv, Honv,Tennv
```

```
FROM [Nhan vien] WHERE Thanhpho<> ALL  
(SELECT Thanhpho FROM [Khach hang])
```

Subquery với các toán tử so sánh

Select Masp, tensp, dongia from [San pham]
Where dongia>ALL (Select dongia from [San pham] where tensp like 'B%')

Select Masp, tensp, dongia from [San pham]
Where dongia>ANY (Select dongia from [San pham] where tensp like 'B%')

Select Masp, tensp, dongia from [San pham]
Where dongia=ANY (Select dongia from [San pham] where tensp like 'B%')

Using EXISTS and NOT EXISTS

- Để kiểm tra xem một truy vấn con có trả về dòng kết quả nào hay không.
- Lượng từ EXISTS (tương ứng NOT EXISTS) trả về giá trị True (tương ứng False) nếu kết quả của truy vấn con có ít nhất một dòng (tương ứng không có dòng nào).
- Điều khác biệt của việc sử dụng EXISTS với hai cách đã nêu ở trên là trong danh sách chọn của truy vấn con có thể có nhiều hơn hai cột.

Example

```
SELECT Mahd, Makh FROM [Hoa don]
WHERE EXISTS
    (SELECT Makh FROM [Khach hang])
```


Using EXISTS and NOT EXISTS

- Subquery dùng với toán tử EXISTS

```
Select * from [Khach Hang] as k  
where NOT EXISTS ( SELECT * from [Hoa don] as h  
                    where k.Makh= h.makh )
```

```
Select * from [Khach Hang] as k  
where EXISTS ( SELECT * from [Hoa don] as h  
               where k.Makh= h.makh )
```

Using EXISTS and NOT EXISTS

- Cho biết những sản phẩm nào chưa bán được dùng 3 cách left outer join, not in và not exists

1) Select * from [San Pham] s

where NOT EXISTS (SELECT * from [Chi Tiet Hoa don] c where s.masp=c.masp)

2) Select * from [San Pham]

where Masp NOT IN (SELECT masp from [Chi Tiet Hoa don])

3) Select * from [San Pham] s Left outer join [Chi Tiet Hoa don] c ON s.Masp=c.Masp WHERE c.Masp is null

Using EXISTS and NOT EXISTS

- Cho biết những sản phẩm nào có tổng số lượng bán được lớn hơn số lượng trung bình bán ra

1) `Select masp, tensp, TongSL=sum(Soluong)`
`From [San Pham] s, [Chi Tiet Hoa Don] c`
`Where s.masp=c.masp`
`Group by c.masp, tensp`
`Having sum(soluong)>(Select AVG(soluong)`
`from [Chi tiet hoa don])`

Nested Subqueries

Nested subqueries are subqueries within other subqueries

Example

```
SELECT Makh, Tenkh FROM [Khach hang]
WHERE Makh in
(SELECT Makh from [Hoa don]
WHERE Mahd in
      (SELECT Mahd FROM [Chi tiet hoa don]
      WHERE masp>3))
```

Correlated Subqueries

- A subquery refers to the parent query
- A subquery is re-evaluated for every iteration of the parent query

Example

```
SELECT Makh,Tenkh, Thanhpho FROM [Khach hang]
WHERE Makh IN
(SELECT Makh FROM [Nhan vien], [hoa don]
WHERE [Hoa don].Manv = [Nhan vien].Manv and [Nhan
vien].Thanhpho =[Khach hang].Thanhpho)
```

Unions – Phép hợp

Toán tử Union để kết nối 2 câu lệnh Select

Cú pháp

```
SELECT statement  
UNION [ALL]  
SELECT statement
```

Ví dụ

```
SELECT c.Thanhpho FROM [Khach hang] c  
UNION  
SELECT e.Thanhpho FROM [Nhan vien] e
```

Unions – Phép hợp

Khi sử dụng toán tử UNION để thực hiện phép hợp, ta cần chú ý các nguyên tắc sau:

- Danh sách cột trong các truy vấn thành phần phải có cùng số lượng.
- Các cột tương ứng trong tất cả các bảng, hoặc tập con bất kỳ các cột được sử dụng trong bản thân mỗi truy vấn thành phần phải cùng kiểu dữ liệu.
- Các cột tương ứng trong bản thân từng truy vấn thành phần của một câu lệnh UNION phải xuất hiện theo thứ tự như nhau. Nguyên nhân là do phép hợp so sánh các cột từng cột một theo thứ tự được cho trong mỗi truy vấn.
- Khi các kiểu dữ liệu khác nhau được kết hợp với nhau trong câu lệnh UNION, chúng sẽ được chuyển sang kiểu dữ liệu cao hơn (nếu có thể được).
- Tiêu đề cột trong kết quả của phép hợp sẽ là tiêu đề cột được chỉ định trong truy vấn đầu tiên.

Unions – Phép hợp

Khi sử dụng toán tử UNION để thực hiện phép hợp, ta cần chú ý các nguyên tắc sau:

- Truy vấn thành phần đầu tiên có thể có INTO để tạo mới một bảng từ kết quả của chính phép hợp.
- Mệnh đề ORDER BY và COMPUTE dùng để sắp xếp kết quả truy vấn hoặc tính toán các giá trị thống kê chỉ được sử dụng ở cuối câu lệnh UNION. Chúng không được sử dụng ở trong bất kỳ truy vấn thành phần nào.
- Mệnh đề GROUP BY và HAVING chỉ có thể được sử dụng trong bản thân từng truy vấn thành phần. Chúng không được phép sử dụng để tác động lên kết quả chung của phép hợp.
- Phép toán UNION có thể được sử dụng bên trong câu lệnh INSERT.
- Phép toán UNION không được sử dụng trong câu lệnh CREATE VIEW.

Lệnh SELECT INTO – Tạo bảng

❑ Ta có thể tạo table mới dựa vào tập kết quả của câu lệnh Select. Table mới có thể là table tạm hay là một table thực sự trong DB.

❑ Cú pháp:

```
SELECT * | ColumnName1, ColumnName2,...  
    INTO TableName  
    FROM Tables  
    WHERE Condition  
    ORDER By SortFieldName  
    GROUP BY FieldGroupName
```

Lệnh SELECT INTO – Tạo bảng

Tạo Table Tạm

```
SELECT Tenkh as Ten, ThanhPho  
INTO #Temp_Customer  
FROM [Khach hang]
```

Xem kết quả

```
Select * From #Temp_Customer
```

Lệnh SELECT INTO – Tạo bảng

Ví dụ

```
SELECT c.Makh As Name, Mahd,  
NgayLapHD
```

```
INTO Customer_Order
```

```
FROM [Khach hang] as c INNER Join [Hoa  
don] As o
```

```
ON c.Makh=o.Makh
```

```
WHERE Month(NgayLapHD) =7
```

Lệnh SELECT INTO – Tạo bảng

Ví dụ: Tạo bảng dữ liệu từ DataBase khác

```
USE SalesDB
```

```
SELECT CompanyName As Name, Phone
```

```
INTO KhachHang
```

```
FROM NorthWind.dbo.Customers
```

Hiệu chỉnh dữ liệu

- Lệnh INSERT
- Lệnh UPDATE
- Lệnh DELETE

Lệnh Insert

- ❑ Thêm một hay nhiều dòng dữ liệu vào bảng hay một view.
- ❑ Lưu ý:
 - Dữ liệu chèn phải đúng kiểu dữ liệu của cột.
 - Không nhập giá trị Null vào cột Not Null.
 - Tuân thủ đúng các toàn vẹn dữ liệu, bất lỗi.
 - Chỉ định danh sách các giá trị ứng với các cột.
 - Cột có thuộc tính Identity thì không cần chỉ định giá trị.
 - Tại một thời điểm chỉ có thể chèn vào một bảng duy nhất.

Lệnh INSERT

❑ Cú pháp 1

**INSERT <TableName> [(Col1, Col2, ...)]
VALUES (Value1, Value2,...)**

❑ Ví dụ 1

INSERT [San pham](Masp, Tensp)
VALUES (123, 'Ice Tea')

Lệnh Insert

Create table t1 (col1 int identity, col2 varchar(30) default('my column default'), col3 timestamp, col4 varchar(40) null);

Insert into t1 (col4) values('Explicit value')

Select * from t1

Insert into t1 (col2,col4) values('Explicit value','Explicit value')

Select * from t1

Insert into t1 (col2) values('Explicit value')

Select * from t1

Insert into t1 default values;

Select * from t1

Lệnh Insert

- ❑ Cú pháp 2: tạo 1 bộ kết quả từ một hay nhiều bảng chèn vào 1 bảng khác

```
INSERT <TableName> [(Col1, Col2, ... )]  
SELECT (Value1, Value2,...)
```

- ❑ Ví dụ 2

```
Create table SPMOI (Masp int, tensp nvarchar(20))
```

```
INSERT SPMOI(Od.Masp, P.Tensp)
```

```
SELECT Od.Masp, Tensp
```

```
FROM [San pham] as P INNER JOIN [Chi tiet  
    hoa don] AS Od ON P. Masp = Od. Masp
```

```
Select * from SPMOI
```

Lệnh UPDATE

- Thay đổi/cập nhật dữ liệu trong một bảng

UPDATE <table_name>

SET <column_name> = <expression>

[FROM <table_list>]

[WHERE <search_condition>

- Ví dụ

```
UPDATE [Chi tiet hoa don] SET  
Thue=Dongia+0.1*Dongia
```

```
WHERE Masp<5
```

```
UPDATE [Chi Tiet Hoa don]
```

```
SET Dongia=
```

```
(SELECT Dongia + Dongia*0.2 FROM [San  
pham])
```

```
Where Masp=2
```

Lệnh DELETE

- ❑ Xóa một hay nhiều dòng trong một bảng hay một truy vấn

DELETE <table_name>

[FROM <table_list>]

[WHERE <search_condition>

- ❑ Ví dụ

```
DELETE FROM [Chi Tiet Hoa Don]  
WHERE Mahd =10148
```

Lệnh TRUNCATE TABLE

- ❑ Lệnh TRUNCATE TABLE dùng để
 - Xóa toàn bộ dữ liệu trong một bảng mà không ghi nhật ký lại
 - Về chức năng hoàn toàn giống như câu lệnh Delete
 - Nhanh hơn câu lệnh delete
 - Không bật các bẫy lỗi trigger

TRUNCATE TABLE *table_name*

- ❑ Ví dụ

TRUNCATE TABLE NewProducts

Merging Data

- Trong SQL Server 2008, có thể thực hiện các thao tác insert, update, hay delete chỉ trong 1 lệnh.
- Lệnh MERGE cho phép kết nối nguồn dữ liệu với bảng/view đích và sau đó thực hiện nhiều action trên bảng đích.

```
MERGE TargetTable
  USING SourceTable
    ON join conditions
  [WHEN Matched
    THEN DML]
  [WHEN NOT MATCHED BY TARGET
    THEN DML]
  [WHEN NOT MATCHED BY SOURCE
    THEN DML]
```

Ví dụ Merging Data để thực thi Insert và Update

dbo.Purchases

ProductID	CustomerID	PurchaseDate
707	11794	2006-08-20
707	15160	2006-08-25
708	18529	2006-08-21
711	11794	2006-08-20
711	19585	2006-08-22
712	14680	2006-08-26
712	21524	2006-08-26
712	19072	2006-08-20
870	15160	2006-08-23
870	11927	2006-08-24
870	18749	2006-08-25

dbo.FactBuyingHabits

ProductID	CustomerID	LastPurchaseDate
707	11794	2006-08-14
707	18178	2006-08-18
864	14114	2006-08-18
866	13350	2006-08-18
866	20201	2006-08-15
867	20201	2006-08-14
869	19893	2006-08-15
870	17151	2006-08-18
870	15160	2006-08-17
871	21717	2006-08-17
871	21163	2006-08-15
871	13350	2006-08-15
873	23381	2006-08-15

Ví dụ Merging Data để thực thi Insert và Update

- Hai bảng trên có 2 hàng giống nhau:
 - Customer 11794 và Product 707
 - Customer 15160 và Product 870
- Dùng lệnh MERGE để cập nhật dữ liệu cho 2 hàng này trong bảng **FactBuyingHabits** với số lượng đặt mua trong bảng **Purchases** (bảng mệnh đề WHEN MATCHED THEN), và chèn tất cả hàng còn lại của **Purchases** vào bảng **FactBuyingHabits** (bảng mệnh đề WHEN NOT MATCHED THEN)

Ví dụ Merging Data để thực thi Insert và Update

MERGE dbo.FactBuyingHabits AS Target

USING (SELECT CustomerID, ProductID, PurchaseDate FROM
dbo.Purchases) AS Source

ON (Target.ProductID = Source.ProductID AND
Target.CustomerID = Source.CustomerID)

WHEN MATCHED THEN

UPDATE SET Target.LastPurchaseDate = Source.PurchaseDate

WHEN NOT MATCHED BY TARGET THEN

INSERT (CustomerID, ProductID, LastPurchaseDate) **VALUES**
(Source.CustomerID, Source.ProductID, Source.PurchaseDate)

Xem bảng

Syntax: Viewing table information

```
sp_help <table_name>
```

```
sp_help constraint <table_name>
```

Example: Viewing table information

```
Sp_help [Khach Hang]
```

```
Sp_help constraint [Khach hang]
```

Xóa bảng

Syntax

```
DROP TABLE <Table_Name>
```

Example

```
DROP TABLE Sanpham
```

View

Nội dung

- ❑ Giới thiệu View
- ❑ Tạo View
- ❑ Sửa View
- ❑ Xóa View
- ❑ Partitioned View

Định nghĩa

- Một khung nhìn (view) có thể được xem như là một **bảng “ảo”** trong cơ sở dữ liệu có nội dung được định nghĩa thông qua một truy vấn (câu lệnh SELECT).
- Một khung nhìn là một tập bao gồm các dòng và các cột.
- Khung nhìn không được xem là một cấu trúc lưu trữ dữ liệu tồn tại trong cơ sở dữ liệu.
- Dữ liệu quan sát được trong khung nhìn được lấy từ các bảng thông qua câu lệnh truy vấn dữ liệu và là **kết quả động** khi view được tham chiếu.

Thuận lợi khi sử dụng view

- ❑ **Bảo mật dữ liệu:** Chỉ cho User xem những gì cần xem nên hạn chế được phần nào việc người sử dụng truy cập trực tiếp dữ liệu.
- ❑ **Đơn giản hoá các thao tác truy vấn dữ liệu:** Một khung nhìn là một đối tượng tập hợp dữ liệu từ nhiều bảng khác nhau vào trong một “bảng”. User có thể thực hiện các yêu cầu truy vấn dữ liệu một cách đơn giản thay vì phải dùng truy vấn phức tạp.
- ❑ **Tập trung và đơn giản hóa dữ liệu:** cung cấp cho người sử dụng những cấu trúc đơn giản, dễ hiểu hơn về dữ liệu trong CSDL đồng thời giúp cho người sử dụng tập trung hơn trên những phần dữ liệu cần thiết.
- ❑ **Độc lập dữ liệu:** người sử dụng có được cái nhìn về dữ liệu độc lập với cấu trúc của các bảng trong CSDL cho dù các bảng cơ sở có bị thay đổi phần nào về cấu trúc.
- ❑ **Dùng để Import, Export**

Thuận lợi khi sử dụng view

MASV	HODEM	TEN	NGAYSINH
0241010001	Ngô Thị Nhật	Anh	Nov 27 1982 ...
0241010002	Nguyễn Thị Ngọc	Anh	Mar 21 1983 ...
0241010003	Ngô Việt	Bắc	May 11 1982 ...
0241010004	Nguyễn Đình	Bình	Oct 6 1982 ...
0241010005	Hồ Đăng	Chiến	Jan 20 1982 ...
0241020001	Nguyễn Tuấn	Anh	Jul 15 1979 ...
0241020002	Trần Thị Kim	Anh	Nov 4 1982 ...
...	...	...	...

Table SINHVIEN

MALOP	TENLOP	
C24101	Toán K24	...
C24102	Tin K24	...
C24103	Lý K24	...
..	...	

Table LOP



MASV	HODEM	TEN	TUOI	TENLOP
0241010001	Ngô Thị Nhật	Anh	22	Toán K24
0241010002	Nguyễn Thị Ngọc	Anh	21	Toán K24
0241010003	Ngô Việt	Bắc	22	Toán K24
0241010004	Nguyễn Đình	Bình	22	Toán K24
0241010005	Hồ Đăng	Chiến	22	Toán K24
0241020001	Nguyễn Tuấn	Anh	25	Tin K24
0241020002	Trần Thị Kim	Anh	22	Tin K24
...	...	...	...	...

view DSSV

Hạn chế khi sử dụng View

- Không bao gồm các mệnh đề COMPUTE hoặc COMPUTE BY.
- Không bao gồm từ khóa INTO.
- Chỉ được dùng ORDER BY khi từ khóa TOP được dùng.
- Không thể tham chiếu quá 1024 cột.
- Không thể kết hợp với câu lệnh T-SQL khác trong cùng một bó lệnh.
- Không thể định nghĩa chỉ mục full text trên View.

Tạo View

Cú pháp

```
CREATE VIEW [<db_name>.] [<owner>.] view_name [(column[ ,...n ])]  
[WITH <view_attribute>[,...n]]  
AS <Select_Statement>  
[WITH CHECK OPTION]  
<view_attribute>::=  
    {ENCRYPTION | SCHEMABINDING}
```

WITH CHECK OPTION: bắt buộc tất cả các lệnh hiệu chỉnh dữ liệu của View phải thỏa mãn các tiêu chuẩn trong câu lệnh Select.

ENCRYPTION: Mã hóa câu lệnh Select tạo ra View.

SCHEMABINDING: Kết View với giản đồ

Tạo View

❑ Ví dụ:

CREATE VIEW vwProducts

AS

SELECT ProductName, UnitPrice,
CompanyName

FROM Suppliers

INNER JOIN Products

ON Suppliers.SupplierID =
Products.SupplierID

Tạo View

Ví dụ

```
CREATE VIEW CTHD AS  
SELECT Orderid, Products.Productid, Productname,  
       Quantity, UnitPrice, ToTal = UnitPrice *Quantity  
FROM Products INNER JOIN [Order Details]  
ON Products.Productid = [Order Details].Productid
```

Nguyên tắc tạo View

- ❑ Tên khung nhìn, tên cột trong View và bảng phải tuân theo qui tắc định danh.
- ❑ Không thể qui định ràng buộc và tạo chỉ mục cho khung nhìn.
- ❑ Câu lệnh SELECT với mệnh đề COMPUTE ... BY không được sử dụng để định nghĩa khung nhìn.
- ❑ Phải đặt tên cho các cột của khung nhìn trong các trường hợp sau:
 - Trong kết quả của câu lệnh SELECT có ít nhất một cột được sinh ra bởi một biểu thức và cột đó không được đặt tiêu đề.
 - Tồn tại hai cột trong kết quả của câu lệnh SELECT có cùng tiêu đề cột.

Nguyên tắc tạo View

❑ Ví dụ 1:

CREATE VIEW dsnv AS

```
SELECT Employees.EmployeeID,FirstName+'  
      '+LastName AS HOTEN,  
DATEDIFF(YY,birthdate,GETDATE()) AS tuoi  
FROM Employees
```

	EmployeeID	HOTEN	tuoi
1	1	Nancy Davolio	68
2	2	Andrew Fuller	64
3	3	Janet Leverling	53
4	4	Margaret Peacock	79
5	5	Steven Buchanan	61
6	6	Michael Suyama	53
7	7	Robert King	56
8	8	Laura Callahan	58
9	9	Anne Dodsworth	50

Nguyên tắc tạo View

❑ Ví dụ 2:

```
CREATE VIEW dsnv (MANV, HOTEN, TUOI) AS  
    SELECT Employees.EmployeeID,FirstName+'  
        '+LastName AS HOTEN,  
        DATEDIFF(YY,birthdate,GETDATE()) AS tuoi  
    FROM Employees
```

	MANV	HOTEN	TUOI
1	1	Nancy Davolio	68
2	2	Andrew Fuller	64
3	3	Janet Leverling	53
4	4	Margaret Peacock	79
5	5	Steven Buchanan	61
6	6	Michael Suyama	53
7	7	Robert King	56
8	8	Laura Callahan	58
9	9	Anne Dodsworth	50

Nguyên tắc tạo View

❑ Ví dụ 3:

CREATE VIEW TuổiNv AS

```
SELECT Employees.EmployeeID,FirstName+'  
      '+LastName AS HOTEN,  
DATEDIFF(YY,birthdate,GETDATE()) AS tuoi  
FROM Employees
```

	EmployeeID	HOTEN	tuoi
1	1	Nancy Davolio	68
2	2	Andrew Fuller	64
3	3	Janet Leverling	53
4	4	Margaret Peacock	79
5	5	Steven Buchanan	61
6	6	Michael Suyama	53
7	7	Robert King	56
8	8	Laura Callahan	58
9	9	Anne Dodsworth	50

Tạo View với *ENCRYPTION*

- ❑ With *ENCRYPTION* : Mã hóa câu lệnh Select tạo ra View.

```
CREATE VIEW vwProducts  
WITH ENCRYPTION  
AS  
SELECT CompanyName, ProductName, UnitPrice  
FROM Suppliers INNER JOIN Products  
ON Suppliers.SupplierID = Products.SupplierID  
  
GO  
  
EXEC sp_helptext vwProducts
```


Tạo View với *SCHEMABINDING*

- ❑ With *SCHEMABINDING*: Kết view với một giản đồ. Khi *SCHEMABINDING* được chỉ định, câu lệnh Select phải chỉ rõ chủ quyền của các bảng, các view. Các hàm được tham chiếu View hay bảng tham gia trong view được tạo với schema không thể xóa trừ phi View đó bị xóa hay thay đổi cơ chế này. Câu lệnh Alter table trên bảng tham gia trong view cũng bị lỗi.

```
CREATE VIEW vwProducts
WITH SCHEMABINDING
AS
    SELECT CompanyName, ProductName, UnitPrice
    FROM dbo.Suppliers INNER JOIN dbo.Products
    ON Suppliers.SupplierID = Products.SupplierID
GO
ALTER TABLE dbo.Products
DROP COLUMN UnitPrice
```

Tạo View với lựa chọn Check

Bắt buộc tất cả các câu lệnh hiệu chỉnh dữ liệu thực thi dựa vào View phải tuyệt đối tôn trọng triệt để đến tập tiêu chuẩn trong câu lệnh Select. Nếu không dùng CHECK, các dòng không thể được hiệu chỉnh. Bất kỳ hiệu chỉnh nào mà sẽ gây ra tình trạng thay đổi đều bị hủy bỏ và một lỗi được hiện ra.

```
CREATE VIEW CustomersCAView AS
```

```
    SELECT * FROM Customers WHERE city='LonDon'
```

```
Select * from CustomersCAView
```

```
GO
```

```
UPDATE CustomersCAView SET city='Anh Quoc' WHERE  
    CustomerID='AROUT'
```

```
Select * from Customers where CustomerID='AROUT'
```

Tạo View với lựa chọn Check

```
CREATE VIEW CustomersCAView1
```

```
AS
```

```
SELECT * FROM Customers WHERE city='LonDon'
```

```
WITH CHECK OPTION
```

```
Select * from CustomersCAView1
```

```
GO
```

```
UPDATE CustomersCAView1 SET city='Anh Quoc'
```

```
WHERE CustomerID='NORTS'
```

Cập nhật, bổ sung và xoá dữ liệu thông qua View

- ❑ **Các thao tác bổ sung, cập nhật và xoá, một khung nhìn phải thoả mãn các điều kiện sau đây:**
 - Trong câu lệnh SELECT định nghĩa khung nhìn không được sử dụng từ khoá DISTINCT, TOP, GROUP BY và UNION.
 - Các thành phần xuất hiện trong danh sách chọn của câu lệnh SELECT phải là các cột trong các bảng cơ sở. Trong danh sách chọn không được chứa các biểu thức tính toán, các hàm gộp.
- ❑ **Các thao tác thay đổi đến dữ liệu thông qua khung nhìn còn phải đảm bảo tính toàn vẹn dữ liệu.**

Cập nhật dữ liệu thông qua View

- ❑ Ví dụ: Xét định nghĩa hai bảng DONVI và NHANVIEN như sau:

```
CREATE TABLE donvi
(  madv INT PRIMARY KEY,
   tendv NVARCHAR(30) NOT NULL,
   dienthoai NVARCHAR(10) NULL
)
CREATE TABLE nhanvien
(  manv NVARCHAR(10) PRIMARY KEY,
   hoten NVARCHAR(30) NOT NULL,
   ngaysinh DATETIME NULL,
   diachi NVARCHAR(50) NULL,
   madv INT FOREIGN KEY
   REFERENCES donvi(madv)
   ON DELETE CASCADE
   ON UPDATE CASCADE
)
```

Cập nhật dữ liệu thông qua View

- ❑ Ví dụ: Xét định nghĩa hai bảng DONVI và NHANVIEN như sau:

Insert into DonVi (Madv, Tendv, DienThoai) values (1,'P.Kinh doanh','822321')

Insert into DonVi (Madv, Tendv, DienThoai) values (2,Tiep thi,'822012')

Insert into nhanvien(manv,hoten,ngaysinh,diachi,madv)
Values('NV01','Tran Van A','3/2/1975','77 Tran Phu',1)

Insert into nhanvien(manv,hoten,ngaysinh,diachi,madv)
Values('NV02','Mai Thi Bich','13/2/1977','17 Nguyen Hue',2)

Insert into nhanvien(manv,hoten,ngaysinh,diachi,madv)
Values('NV03','Le Van Ha','3/2/1973','12 Tran Phu',2)

MADV	TENDV	DIENTHOAI
1	P. Kinh doanh	822321
2	P. Tiep thi	822012

Bảng DONVI

MANV	HOTEN	NGAYSINH	DIACHI	MADV
NV01	Tran Van A	1975-02-03 00:00:00	77 Tran Phu	1
NV02	Mai Thi B	1977-05-04 00:00:00	34 Nguyen Hue	2
NV03	Nguyen Van C	NULL	NULL	2

Bảng NHANVIEN

Cập nhật, bổ sung và xóa dữ liệu thông qua View

```
CREATE VIEW nv1
```

```
AS
```

```
SELECT manv,hoten,madv FROM nhanvien
```

```
GO
```

```
INSERT INTO nv1 VALUES('NV04','Le Thi D',1)
```

MANV	HOTEN	NGAYSINH	DIACHI	MADV
NV01	Tran Van A	1975-02-03 00:00:00	77 Tran Phu	1
NV02	Mai Thi B	1977-05-04 00:00:00	34 Nguyen Hue	2
NV03	Nguyen Van C	NULL	NULL	2
NV04	Le Thi D	NULL	NULL	1

Bản ghi mới



```
DELETE FROM nv1 WHERE manv='NV04'
```

Cập nhật dữ liệu thông qua View

- ❑ Nếu câu lệnh SELECT có sự xuất hiện của biểu thức tính toán đơn giản, thao tác bổ sung dữ liệu thông qua khung nhìn không thể thực hiện được. Tuy nhiên, thao tác cập nhật và xóa dữ liệu vẫn có thể có khả năng thực hiện được (trừ cột là một biểu thức tính toán).

- ❑ Ví dụ : Xét khung nhìn NV2 được định nghĩa như sau:

```
CREATE VIEW nv2
```

```
AS
```

```
    SELECT manv,hoten,YEAR(ngaysinh) AS namsinh,madv FROM nhanvien
```

```
GO
```

```
INSERT INTO nv2(manv,hoten,madv) VALUES('NV05','Le Van E',1) -Lỗi
```

```
GO
```

```
UPDATE nv2 SET hoten='Le Thi X' WHERE manv='NV04' -Thực hiện được
```

```
GO
```

```
DELETE FROM nv2 WHERE manv='NV04' -Thực hiện được
```


Cập nhật dữ liệu thông qua View

- ❑ Nếu khung nhìn được tạo ra từ một phép nối (trong hoặc ngoài) trên nhiều bảng, ta có thể thực hiện được thao tác bổ sung hoặc cập nhật dữ liệu nếu thao tác này chỉ có tác động đến đúng một bảng cơ sở (câu lệnh DELETE không thể thực hiện được trong trường hợp này).

- ❑ Ví dụ: Với khung nhìn được định nghĩa như sau:

```
CREATE VIEW nv3
```

```
AS
```

```
SELECT manv,hoten,ngaysinh, diachi,nhanvien.madv AS  
noilamviec, donvi.madv,tendv,dienthoai FROM nhanvien FULL  
OUTER JOIN donvi ON nhanvien.madv=donvi.madv
```

```
GO
```

```
--Thêm vào bảng NHANVIEN
```

```
INSERT INTO nv3(manv,hoten,noilamviec) VALUES('NV05','Le Van  
E',1)
```

```
--Thêm vào bảng DONVI
```

```
INSERT INTO nv3(madv,tendv) VALUES(3,'P. Ke toan')
```

Bổ sung dữ liệu thông qua View

❑ Cú pháp:

***ALTER VIEW tên_khung_nhìn [(danh_sách_tên_cột)]
AS***

Câu_lệnh_SELECT

❑ Ví dụ: Ta định nghĩa khung nhìn như sau:

CREATE VIEW viewDV

AS

SELECT manv,hoten,tendv

FROM donvi INNER JOIN nhanvien ON donvi.madv=nhanvien.madv

WHERE tendv='P.Kinh doanh'

Select * from viewDV

Drop view viewDV

ALTER VIEW viewDV

AS

SELECT manv, hoten, tendv

FROM donvi INNER JOIN nhanvien ON donvi.madv=nhanvien.madv

WHERE tendv='Tiep Thi'

Xóa View

❑ Cú pháp:

DROP VIEW tên_khung_nhìn

❑ Nếu một khung nhìn bị xoá, toàn bộ những quyền đã cấp phát cho người sử dụng trên khung nhìn cũng đồng thời bị xoá. Do đó, nếu ta tạo lại khung nhìn thì phải tiến hành cấp phát lại quyền cho người sử dụng.

❑ Ví dụ: DROP VIEW viewDV

Đổi tên Views

❑ Đổi tên Views:

Cú pháp:

`sp_rename old_viewname, new_viewname`

Ví dụ : `Sp_rename CTHD, ChiTietHD`

❑ Xác nhận Views:

Cú pháp:

`sp_helptext viewname`

Ví dụ : `Sp_helptext ChitietHD`

Các loại Views

- ❑ **Standard View**
- ❑ **Indexed View**
- ❑ **Partitioned View**

Các loại Views

□ Standard View

- Kết hợp dữ liệu từ nhiều bảng tùy theo mục đích sử dụng.
- Các lợi ích khi sử dụng view:
 - ✓ Cung cấp dữ liệu thích hợp cho người dùng
 - ✓ Che giấu sự phức tạp của dữ liệu
 - ✓ Kết hợp dữ liệu từ các nguồn không đồng nhất
 - ✓ View được dùng như 1 cơ chế bảo mật (security mechanism) bằng cách cho phép người dùng truy xuất dữ liệu thông qua view mà không cấp cho người dùng quyền được truy xuất trực tiếp dữ liệu từ bảng gốc.

Các loại Views

□ Standard View

EmployeeMaster table

EmployeeID	FirstName	AddressID	ShiftID	LastName	MiddleName	SSN	...
1	Sheri	1	1	Nowmer	E	245797967	...
2	Derrick	2	1	Whelply	R	509647174	...
3	Michael	3	1	Spence	C	42487730	...
4	Maya	4	1	Gutierrez	Y	56920285	...
5	Roberta	5	1	Damstra	B	695256908	...

View

FirstName	LastName	Description
Sheri	Nowmer	Engineering
Derrick	Whelply	Engineering
Michael	Spence	Engineering

Department Table

DepartmentID	Description	rowguid
1	Engineering	3FFD2603-EB6E-43B2-A8EF-C4F5C3064026
2	Tool Design	AE948718-D4BF-40E0-8ECD-2D9F4A0B211E
3	Sales	702C0EE3-03E6-4F95-9AB8-99F4F25921F3
4	Marketing	3E3C4476-B9EC-43CB-AA12-1E7A140A71A4
5	Purchasing	D6C63691-93B5-4F43-AD88-34B6B9A3C4A3

Các loại Views

❑ Indexed View

- Là view đã được hiện thực hóa (materialized), nghĩa là view đã được tính toán thực thi và lưu trữ như 1 bảng thực. View có 1 chỉ mục clustered duy nhất.
- Lý do sử dụng indexed view:

Nếu 1 view tham chiếu đến 1 truy vấn phức tạp thì chi phí để tạo dựng bộ kết quả cho truy vấn khi sử dụng view rất lớn. Để cải thiện việc thực thi, cần tạo chỉ mục cho view.

- View chỉ mục không thích hợp cho những bảng dữ liệu hay cập nhật chỉnh sửa

Các loại Views

❑ Indexed View

- Để tạo clustered index cho 1 view, cần phải đáp ứng các yêu cầu sau:
 - Không được tham chiếu đến các view khác, chỉ từ bảng gốc. Tất cả bảng gốc này phải cùng 1 database và có cùng 1 owner.
 - View phải được tạo ra với tùy chọn SCHEMABINDING.
 - Các hàm người dùng được tham chiếu đến trong view phải được ra với tùy chọn SCHEMABINDING.
 - Bảng và các hàm người dùng phải được tham chiếu bởi tên gồm 2 thành phần.

Các loại Views

❑ Indexed View

```
CREATE VIEW Sales.vOrders
WITH SCHEMABINDING
AS
    SELECT SUM(UnitPrice*OrderQty*(1.00-UnitPriceDiscount)) AS Revenue,
           OrderDate, ProductID, COUNT_BIG(*) AS COUNT
    FROM Sales.SalesOrderDetail AS od, Sales.SalesOrderHeader AS o
    WHERE od.SalesOrderID = o.SalesOrderID
    GROUP BY OrderDate, ProductID;
```

```
--Create an index on the view.
CREATE UNIQUE CLUSTERED INDEX IDX_V1
    ON Sales.vOrders (OrderDate, ProductID);
```

Các loại Views

❑ Indexed View

Create View HDKH
WITH SCHEMABINDING
AS

Select orderdate,COUNT(*) As ToTal
From [Customers] c , Orders o
Where c.CustomerID = o.CustomerID
Group by OrderDate

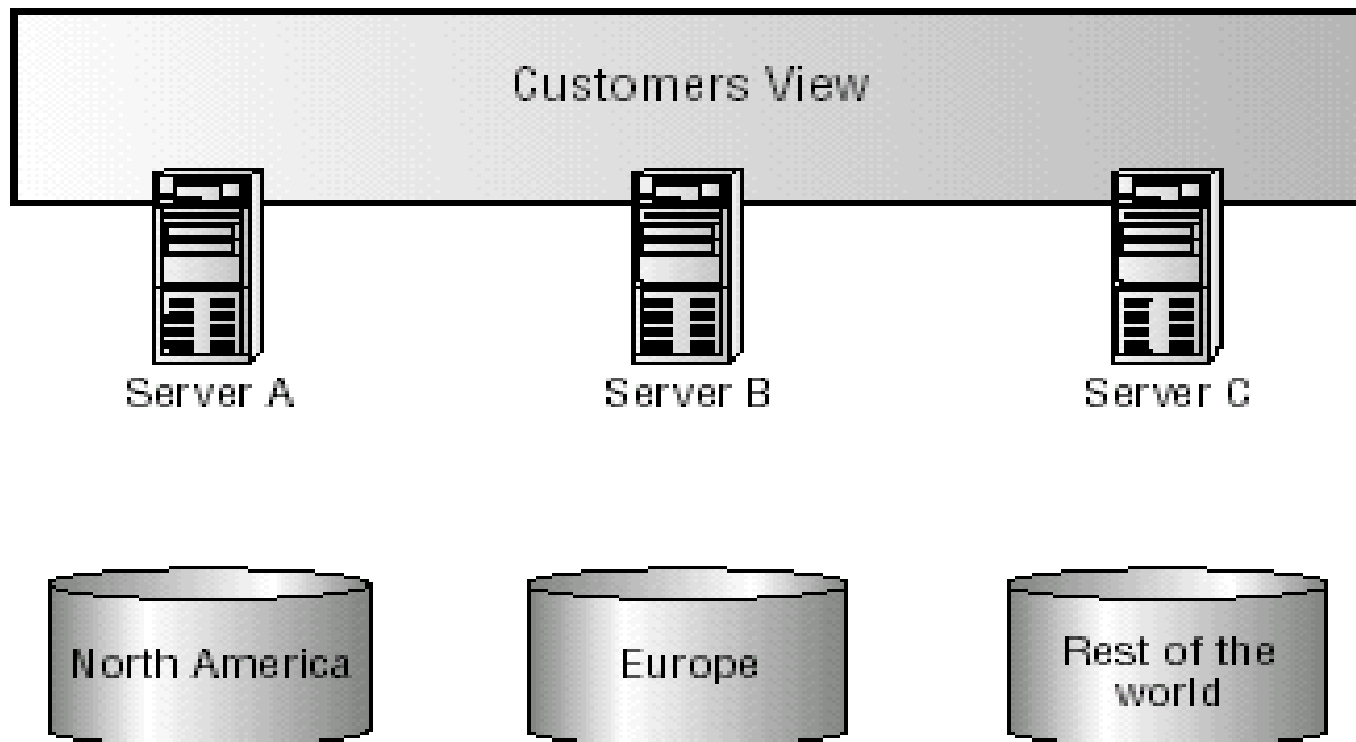
Create UNIQUE CLUSTERED INDEX IDX_V1 ON
SalesOrder(orderdate,Productid);

Các loại Views

❑ Indexed View

```
--This query can use the indexed view even though the view is
--not specified in the FROM clause.
SELECT SUM(UnitPrice*OrderQty*(1.00-UnitPriceDiscount)) AS Rev,
       OrderDate, ProductID
FROM Sales.SalesOrderDetail AS od
      JOIN Sales.SalesOrderHeader AS o ON od.SalesOrderID=o.SalesOrderID
      AND ProductID BETWEEN 700 and 800
      AND OrderDate >= CONVERT(datetime,'05/01/2002',101)
GROUP BY OrderDate, ProductID
ORDER BY Rev DESC;
```

Partitioned Views



Partitioned Views

- Các bảng tham gia Partition view phải có cấu trúc giống nhau.
- Có một cột có check constraint với phạm vi của ràng buộc CHECK ở mỗi bảng là khác nhau.
- Tạo View bằng cách kết các dữ liệu bằng từ khóa UNION ALL.
- Cột là NOT NULL.
- Cột là một phần khóa chính của table.
- Không có cột tính toán.
- Chỉ có duy nhất một ràng buộc CHECK tồn tại trong một cột.
- Bảng không thể có chỉ mục trong các cột tính toán.

Partitioned Views

Ví dụ:

```
CREATE VIEW Customers
AS
SELECT * FROM
ServerA.MyCompany.dbo.CustomersAmerica
UNION ALL
SELECT * FROM
ServerB.MyCompany.dbo.CustomersEurope
UNION ALL
SELECT * FROM
ServerC.MyCompany.dbo.CustomersAsia
```

Partitioned Views

Ví dụ

Create Table KH_BAC

```
(Makh int, TenKh Nchar(30),  
  Khuvuc Nvarchar(30) NOT NULL CHECK (Khuvuc='Bac bo'),  
  PRIMARY KEY (Makh, Khuvuc)  
)
```

Create Table KH_TRUNG

```
(Makh int, TenKh Nchar(30),  
  Khuvuc Nvarchar(30) NOT NULLCHECK (Khuvuc='Trung bo'),  
  PRIMARY KEY (Makh, Khuvuc))
```


Partitioned Views

Create Table KH_NAM

(Makh int, TenKh Nchar(30),

Khuvuc Nvarchar(30) NOT NULL CHECK
(Khuvuc='Nam bo'),

PRIMARY KEY (Makh, Khuvuc)

)

Partitioned Views

Create View Khachhang
AS

```
        Select * From KH_BAC  
UNION ALL  
        Select * From KH_TRUNG  
UNION ALL  
        Select * From KH_NAM
```

INSERT Khachhang VALUES (1, 'CDCN4','Nam Bo')

SELECT * FROM KH_Nam

Hiệu chỉnh dữ liệu thông qua Partitioned View

- Tất cả các cột phải có giá trị ngay cả cột chấp nhận Null và cột có giá trị Default.
- Từ khóa Default không được sử dụng trong câu lệnh Insert, Update.
- Phải có giá trị đúng của cột có ràng buộc CHECK.
- Câu lệnh INSERT không cho phép nếu bảng thành viên có cột có thuộc tính Identity, cột Timestamp.
- Không Insert, Update hay Delete nếu có một kết self-join trong cùng View hay bảng thành viên.
- Khi dùng lệnh Delete ta có thể xóa các mẫu tin trong bảng thành viên thông qua View.